karpathy / **microgpt.py**

Last active 47 minutes ago • Report abuse

Code | Revisions 13 | Stars 5,000+ | Forks 2,129

microgpt

microgpt.py

```python
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
random.shuffle(docs)
print(f"num docs: {len(docs)}")

# Let there be a Tokenizer to translate strings to sequences of integers ("tokens") and ba
uchars = sorted(set(''.join(docs))) # unique characters in the dataset become token ids 0.
BOS = len(uchars) # token id for a special Beginning of Sequence (BOS) token
vocab_size = len(uchars) + 1 # total number of unique tokens, +1 is for BOS
print(f"vocab size: {vocab_size}")

# Let there be Autograd to recursively apply the chain rule through a computation graph
class Value:
    __slots__ = ('data', 'grad', '_children', '_local_grads') # Python optimization for me

    def __init__(self, data, children=(), local_grads=()):
        self.data = data                # scalar value of this node calculated during forw
        self.grad = 0                   # derivative of the loss w.r.t. this node, calcula
        self._children = children       # children of this node in the computation graph
        self._local_grads = local_grads # local derivative of this node w.r.t. its childre
```

```python

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1, 1))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))

    def __pow__(self, other): return Value(self.data**other, (self,), (other * self.data**
    def log(self): return Value(math.log(self.data), (self,), (1/self.data,))
    def exp(self): return Value(math.exp(self.data), (self,), (math.exp(self.data),))
    def relu(self): return Value(max(0, self.data), (self,), (float(self.data > 0),))
    def __neg__(self): return self * -1
    def __radd__(self, other): return self + other
    def __sub__(self, other): return self + (-other)
    def __rsub__(self, other): return other + (-self)
    def __rmul__(self, other): return self * other
    def __truediv__(self, other): return self * other**-1
    def __rtruediv__(self, other): return other * self**-1

    def backward(self):
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._children:
                    build_topo(child)
                topo.append(v)
        build_topo(self)
        self.grad = 1
        for v in reversed(topo):
            for child, local_grad in zip(v._children, v._local_grads):
                child.grad += local_grad * v.grad

# Initialize the parameters, to store the knowledge of the model
n_layer = 1     # depth of the transformer neural network (number of layers)
n_embd = 16     # width of the network (embedding dimension)
block_size = 16 # maximum context length of the attention window (note: the longest name i
n_head = 4      # number of attention heads
head_dim = n_embd // n_head # derived dimension of each head
matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0, std)) for _ in range(nin)] fo
state_dict = {'wte': matrix(vocab_size, n_embd), 'wpe': matrix(block_size, n_embd), 'lm_he
for i in range(n_layer):
    state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
```

```
    state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_embd)
params = [p for mat in state_dict.values() for row in mat for p in row] # flatten params i
print(f"num params: {len(params)}")

# Define the model architecture: a function mapping tokens and parameters to logits over w
# Follow GPT-2, blessed among the GPTs, with minor differences: layernorm -> rmsnorm, no b
def linear(x, w):
    return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in w]

def softmax(logits):
    max_val = max(val.data for val in logits)
    exps = [(val - max_val).exp() for val in logits]
    total = sum(exps)
    return [e / total for e in exps]

def rmsnorm(x):
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]

def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id] # token embedding
    pos_emb = state_dict['wpe'][pos_id] # position embedding
    x = [t + p for t, p in zip(tok_emb, pos_emb)] # joint token and position embedding
    x = rmsnorm(x) # note: not redundant due to backward pass via the residual connection

    for li in range(n_layer):
        # 1) Multi-head Attention block
        x_residual = x
        x = rmsnorm(x)
        q = linear(x, state_dict[f'layer{li}.attn_wq'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear(x, state_dict[f'layer{li}.attn_wv'])
        keys[li].append(k)
        values[li].append(v)
        x_attn = []
        for h in range(n_head):
            hs = h * head_dim
            q_h = q[hs:hs+head_dim]
            k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
            v_h = [vi[hs:hs+head_dim] for vi in values[li]]
            attn_logits = [sum(q_h[j] * k_h[t][j] for j in range(head_dim)) / head_dim**0.
            attn_weights = softmax(attn_logits)
            head_out = [sum(attn_weights[t] * v_h[t][j] for t in range(len(v_h))) for j in
            x_attn.extend(head_out)
        x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
        x = [a + b for a, b in zip(x, x_residual)]
        # 2) MLP block
```

```
        x_residual = x
        x = rmsnorm(x)
        x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
        x = [xi.relu() for xi in x]
        x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
        x = [a + b for a, b in zip(x, x_residual)]

    logits = linear(x, state_dict['lm_head'])
    return logits

# Let there be Adam, the blessed optimizer and its buffers
learning_rate, beta1, beta2, eps_adam = 0.01, 0.85, 0.99, 1e-8
m = [0.0] * len(params) # first moment buffer
v = [0.0] * len(params) # second moment buffer

# Repeat in sequence
num_steps = 1000 # number of training steps
for step in range(num_steps):

    # Take single document, tokenize it, surround it with BOS special token on both sides
    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)

    # Forward the token sequence through the model, building up the computation graph all
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses) # final average loss over the document sequence. May your

    # Backward the loss, calculating the gradients with respect to all model parameters
    loss.backward()

    # Adam optimizer update: update the model parameters based on the corresponding gradie
    lr_t = learning_rate * (1 - step / num_steps) # linear learning rate decay
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))
        p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
        p.grad = 0

    print(f"step {step+1:4d} / {num_steps:4d} | loss {loss.data:.4f}", end='\r')
```

```python

# Inference: may the model babble back to us
temperature = 0.5 # in (0, 1], control the "creativity" of generated text, low to high
print("\n--- inference (new, hallucinated names) ---")
for sample_idx in range(20):
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    token_id = BOS
    sample = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax([l / temperature for l in logits])
        token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
        if token_id == BOS:
            break
        sample.append(uchars[token_id])
```

Load earlier comments...

busfahrer commented 2 weeks ago • edited

Here's my hack of adding MoE to microgpt, as a learning exercise:

https://gist.github.com/busfahrer/e5f4ca6b81b127cd5eb1a99e20750622

I tried to change as little as possible, and kept the style similarly terse as the original. Since I'm still learning, I'm happy about any comments!

edit:
I've once heard the lisp implementation in lisp being called "Maxwell's equations of software". To me, microgpt is the "Maxwell's equations of LLMs".
(Link to the lisp quote/article: https://www.righto.com/2008/07/maxwells-equations-of-software-examined.html)

smimram commented 2 weeks ago

Here is my re-implementation in OCaml (this was quite a nice exercise in order to understand this more in depth):
https://github.com/smimram/ocaml-microgpt/

rupeshs commented 2 weeks ago • edited

> Here is my re-implementation in OCaml (this was quite a nice exercise in order to understand this more in depth): https://github.com/smimram/ocaml-microgpt/

@smimram Would you be interested in creating a PR to add this to the Awesome MicroGPT list? It would be a great addition. https://github.com/rupeshs/awesome-microgpts

kaishaoshao commented 2 weeks ago

> https://github.com/assassindesign/microgptjs
>
> 使用nodejs+ES5语法的microgpt实现，你甚至可以用他训练写诗 f0ec2a

```
11  廖多瑗
12  江聪磊
13  钟明寒
14  刘有枝
15  冯康风
16  蒋健荷
17  冯丹豆
18  蒋怡岁
19  阎龙芸
20  叶大谦
21  薛柏敬
22  蔡顺
23  宋波绍
24  秦薇
25  卢磊禄
26  程莎金
```

终端 问题 36 输出 MEMORY 端口 调试控制台 XRTOS GITLENS

```
➜  |d2l_new|microgpt git:(main {1} U:2) ✗ python ./microgpt.py
num docs: 30000
vocab size: 413
num params: 16544
step 1000 / 1000 | loss 4.2965
--- inference (new, hallucinated names) ---
sample  1: 丁健
sample  2: 邱真
sample  3: 张豪
sample  4: 许维
sample  5: 潘颖
sample  6: 余昏
sample  7: 宋红
sample  8: 程麦
sample  9: 刘辰
sample 10: 阎禄
sample 11: 刘雁
sample 12: 刘风
sample 13: 杜麦
sample 14: 贾荣
sample 15: 汪日
sample 16: 郝静
sample 17: 傅纨
sample 18: 姚长
sample 19: 张佳
sample 20: 崔孙
➜  |d2l_new|microgpt git:(main {1} U:2) ✗
```

I checked the Chinese characters and they still need adjustment. There's still a problem with how I generate

Chinese names; it's just a random combination.

qwertyuu commented 2 weeks ago

@**rupeshs** you never disappoint! Cool to see you here :)

mplekh commented 2 weeks ago

NEURAL DOOM II: ARENA
Now both sides are neural-network controlled - enemy and player AIs train live in your tab, no server, no frameworks, just raw JS autograd.
Both networks learn from a heuristic teacher every frame, the teacher uses BFS pathfinding to navigate around walls, so both AIs learn obstacle avoidance, not just line-of-sight beelining.
The 192-pixel visual input to the transformer is a vestige of an earlier RL harness (A2C policy gradient) that didn't converge, online per-frame RL with a ~20K param model and noisy rewards just spins in circles. Switching to supervised learning from the heuristic teacher made it work immediately, but the teacher only uses game-state features, so the visual pixels are dead weight the model learns to ignore. They're kept because they look cool on the HUD.

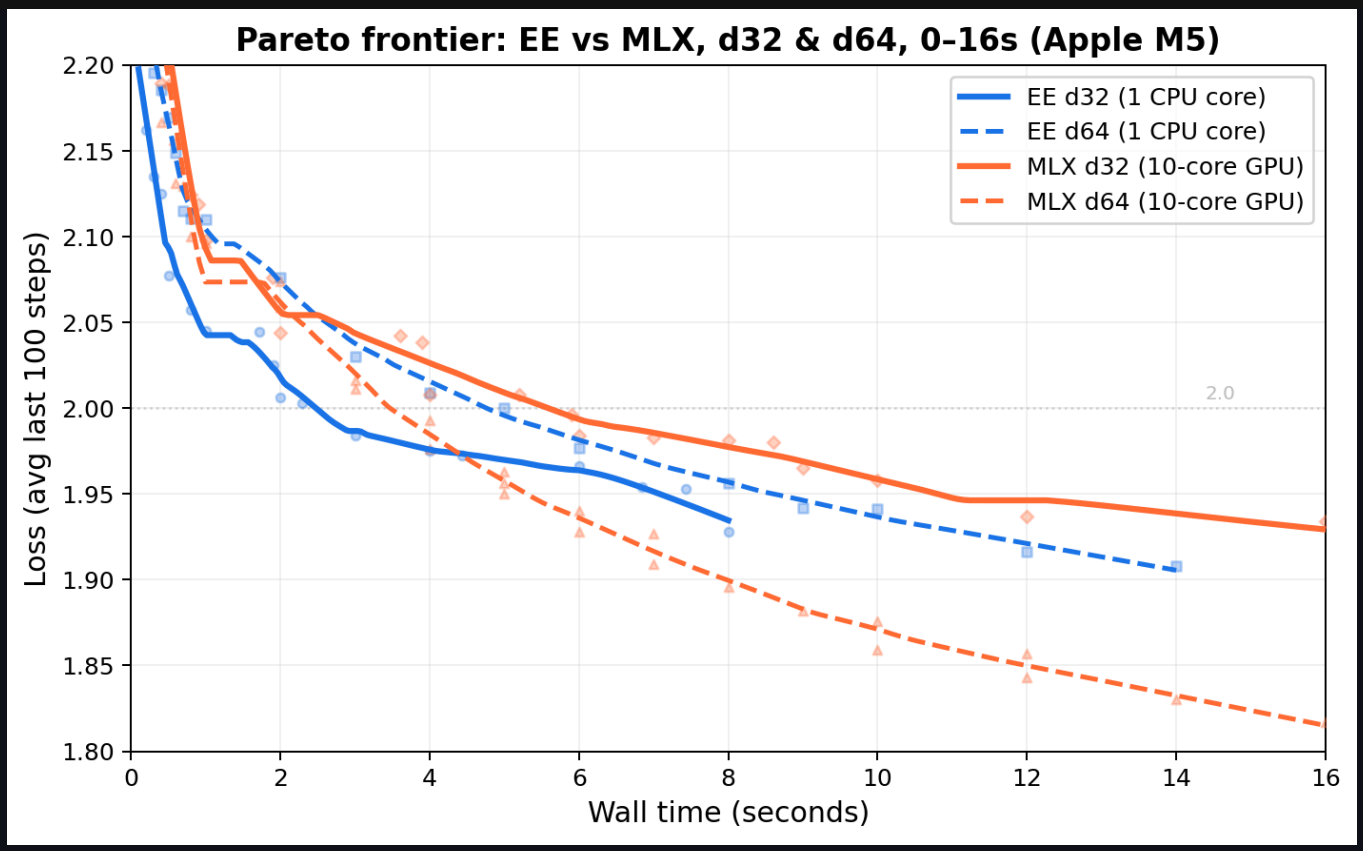
rupeshs commented 2 weeks ago • edited

> @**rupeshs** you never disappoint! Cool to see you here :)

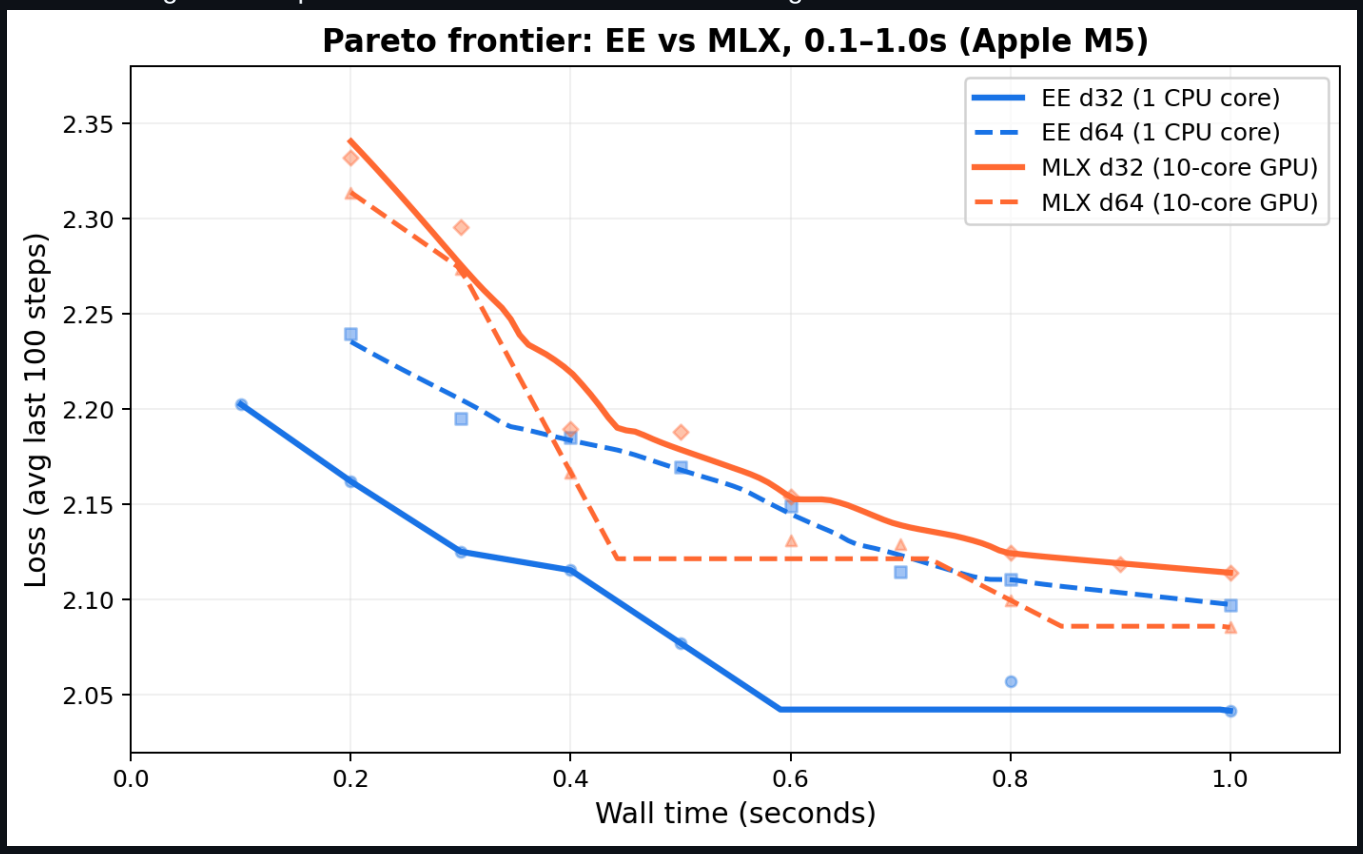
@**qwertyuu** :D

Entrpi commented 2 weeks ago • edited

I got swallowed up by working on other projects, but I had done more testing since publishing my work. I don't want to leave these charts rotting on my local drive indefinitely, so some further testing on EEmicroGPT:



The advantage is most pronounced in the first second of training:



Given how quickly you can iterate with total training times below 1s, I was able to do fairly exhaustive sweeps to produce that chart. That's what's really fun about this implementation, and I do intend to write more about the value of iterating on research at an interactive pace in the future.

ariannamethod commented 2 weeks ago

microreasoning.py = (microgpt.py - charlevel - names.txt) + BPE on input + word-level on output + 1984.txt (that contains exactly 1984 words, 'cause we all should know in which time we live).
output coherent even in weightless mode. you can put names.txt instead: the engine will adapt.

when trained: 19.6M params. 8-layer Resonance engine (RoPE + multi-head attention + RRPRAM + SwiGLU + Dario Equation). the soul thinks in subwords, the mouth speaks real words. the engine detects the most charged word in your prompt and makes 12 associative steps from it:

> "what is consciousness"
> consciousness → ash → prisoner → moss → island → hero → ink → sowing → fog → oar → salt → seed

> "darkness eats the city"
> darkness → fog → hawk → brass → candle → burn → sing → landing → sand → raft → loss → boat

gist: https://gist.github.com/ariannamethod/63d9313f992cdc7fa98309d43ecfe4a9
repo: https://github.com/ariannamethod/1984
weights: https://github.com/ariannamethod/1984/blob/main/weights/penelope.bin

jimenezyesenia718-sketch commented 2 weeks ago

```
gui.ResetOnSpawnlocal player = game.Players.LocalPlayer
local char = player.Character or player.CharacterAdded:Wait()
local humanoidRootPart = char:WaitForChild("HumanoidRootPart")

local savedPos = nil
local flying = false
local noclip = false

-- GUI
local gui = Instance.new("ScreenGui")
gui.Name = "ErikHub"
gui.ResetOnSpawn = false
gui.Parent = player:WaitForChild("PlayerGui")

local frame = Instance.new("Frame")
frame.Size = UDim2.new(0,200,0,160)
frame.Position = UDim2.new(0.4,0,0.3,0)
frame.BackgroundTransparency = 0.3
frame.BackgroundColor3 = Color3.fromRGB(20,20,20)
frame.Active = true
frame.Draggable = true
frame.Parent = gui
```

```
local title = Instance.new("TextLabel")
title.Size = UDim2.new(1,0,0,30)
title.Text = "Erik"
title.BackgroundTransparency = 1
title.TextColor3 = Color3.new(1,1,1)
title.Font = Enum.Font.GothamBold
title.TextScaled = true
title.Parent = frame

local function createButton(name,posY)
local btn = Instance.new("TextButton")
btn.Size = UDim2.new(0.8,0,0,25)
btn.Position = UDim2.new(0.1,0,0,posY)
btn.Text = name
btn.BackgroundColor3 = Color3.fromRGB(40,40,40)
btn.TextColor3 = Color3.new(1,1,1)
btn.Parent = frame
return btn
end

local tp = createButton("TP",40)
local tp2 = createButton("TP2",70)
local tras = createButton("tras",100)
local fli = createButton("fli",130)

-- Guardar posición
tp.MouseButton1Click:Connect(function()
savedPos = humanoidRootPart.Position
end)

-- Teleport a posición guardada
tp2.MouseButton1Click:Connect(function()
if savedPos then
humanoidRootPart.CFrame = CFrame.new(savedPos + Vector3.new(0,3,0))
end
end)
```

-- Noclip (traspasar paredes)
tras.MouseButton1Click:Connect(function()
noclip = not noclip
while noclip do
for _,v in pairs(char:GetDescendants()) do
if v:IsA("BasePart") then
v.CanCollide = false
end
end
task.wait()
end
end)

-- Volar simple
fli.MouseButton1Click:Connect(function()
flying = not flying

```
if flying then
    local bodyVelocity = Instance.new("BodyVelocity")
    bodyVelocity.MaxForce = Vector3.new(1e5,1e5,1e5)
    bodyVelocity.Parent = humanoidRootPart

    while flying do
        bodyVelocity.Velocity = workspace.CurrentCamera.CFrame.LookVector * 50
        task.wait()
    end

    bodyVelocity:Destroy()
end
```

end)

jet10000 commented 2 weeks ago

> NEURAL DOOM II: ARENA Now both sides are neural-network controlled - enemy and player AIs train live in your tab, no server, no frameworks, just raw JS autograd. Both networks learn from a heuristic teacher every frame, the teacher uses BFS pathfinding to navigate around walls, so both AIs learn obstacle avoidance, not just line-of-sight beelining. The 192-pixel visual input to the transformer is a vestige of an earlier RL harness (A2C policy gradient) that didn't converge, online per-frame RL with a ~20K param model and noisy rewards just spins in circles. Switching to supervised learning from the heuristic teacher made it work immediately, but the teacher only uses game-state features, so the visual pixels are dead weight the model learns to ignore. They're kept because they look cool on the HUD.

👍

zaunere commented last week

Incredible - Dropping the Time-to-Information; micro domain near instant uptake and inference can be done !

lexpank commented last week

Might be too much, but this had to be done: microgpt.cu - for everyone who loves GPU ❤️

mplekh commented last week

> Might be too much, but this had to be done: microgpt.cu - for everyone who loves GPU ❤️

Nice and clean, thanks! Is there room to optimize the kernel launches? I'm guessing at block_size=16 / n_embd=16 the GPU is mostly twiddling its thumbs waiting on launch overhead, so there's probably no point. But if you bumped it to 64/64, would it start to be worth it? I've benched on i9-10900X CPU + GTX1070 GPU against rust-microgpt and rust-matrixmicrogpt. At block_size=64 / n_embd=64 and 1000 steps, microgpt.cu time is 5.2s, rust-microgpt - 5.6s, rust-matrixmicrogpt - 1.8s

grahamannett commented last week

Here is a minimal extension of microgpt to include attention residuals from MoonshotAI (keeping to 1 layer for comparison but having only 1 layer may be questionable for what impact the residuals will really have):

https://github.com/grahamannett/micro-attention-residuals/blob/main/micro-full-attention-residuals.py

lexpank commented last week

> Nice and clean, thanks! Is there room to optimize the kernel launches? I'm guessing at block_size=16 / n_embd=16 the GPU is mostly twiddling its thumbs waiting on launch overhead, so there's probably no point. But if you bumped it to 64/64, would it start to be worth it? I've benched on i9-10900X CPU + GTX1070 GPU against rust-microgpt and rust-matrixmicrogpt. At block_size=64 / n_embd=64 and 1000 steps, microgpt.cu time is 5.2s, rust-microgpt - 5.6s, rust-matrixmicrogpt - 1.8s

Will try that in the next revision, thanks 😉

mplekh commented last week

I've upgraded rust-matrixmicrogpt model to use FlashAttention, 1.4x speedup on block_size=16 / n_embd=16 (benched with 50000 steps, takes 10s on i9-10900x) and 1.2x speedup at t block_size=64 / n_embd=64 (i've bundled input2.txt for benching this config)

mplekh commented last week

> Here is a minimal extension of microgpt to include attention residuals from MoonshotAI (keeping to 1 layer for comparison but having only 1 layer may be questionable for what impact the residuals will really have):
>
> https://github.com/grahamannett/micro-attention-residuals/blob/main/micro-full-attention-residuals.py

Thanks for sharing! Indeed with only 1 layer we have here, there's nothing meaningful to attend over (just the embedding).
What makes sense here is mHC: even with 1 transformer layer, the 2 sublayers (attention + MLP) benefit from separate residual pathways via stream mixing/distribution.
So I've upgraded rust-matrixmicrogpt to use mHC in residual path. Performance boost is amazing: 50K steps with block_size=16 / n_embd=16 now takes 5.6s - 2x speedup; and in 64/64 case it's 1.5x faster

Chrisbryan17 commented 5 days ago

This is awesome: tiny, readable, and still a complete GPT training/inference stack.

I used it as a springboard for a dependency-free single-file GPT variant aimed at removing the main bottlenecks here: scalar autograd overhead and token-by-token training. Mine keeps the decoder-only transformer structure but swaps in explicit backward, sequence-level training, fused QKV, tied embeddings, optional ALiBi/QK-Norm, ReLU², cached decode, and production-style checkpoint/test scaffolding.

Gist: https://gist.github.com/Chrisbryan17/5f2ea133583160085a83b1eea9c141b6

Appreciate you putting artifacts like this out in public.

Chrisbryan17 commented 5 days ago

> This is awesome: tiny, readable, and still a complete GPT training/inference stack.
>
> I used it as a springboard for a dependency-free single-file GPT variant aimed at removing the main bottlenecks here: scalar autograd overhead and token-by-token training. Mine keeps the decoder-only transformer structure but swaps in explicit backward, sequence-level training, fused QKV, tied embeddings, optional ALiBi/QK-Norm, ReLU², cached decode, and production-style checkpoint/test scaffolding.

> Gist: https://gist.github.com/Chrisbryan17/5f2ea133583160085a83b1eea9c141b6
>
> Appreciate you putting artifacts like this out in public.

Added another file to my gist inspired by this: a <200 LoC dependency-free GPT-style variant. (117 LoC)

I tried to keep the “single tiny file” spirit, but push harder on the main bottlenecks:
explicit backward, sequence-level training, fused QKV, tied embeddings, ALiBi-style bias, and ReLU².

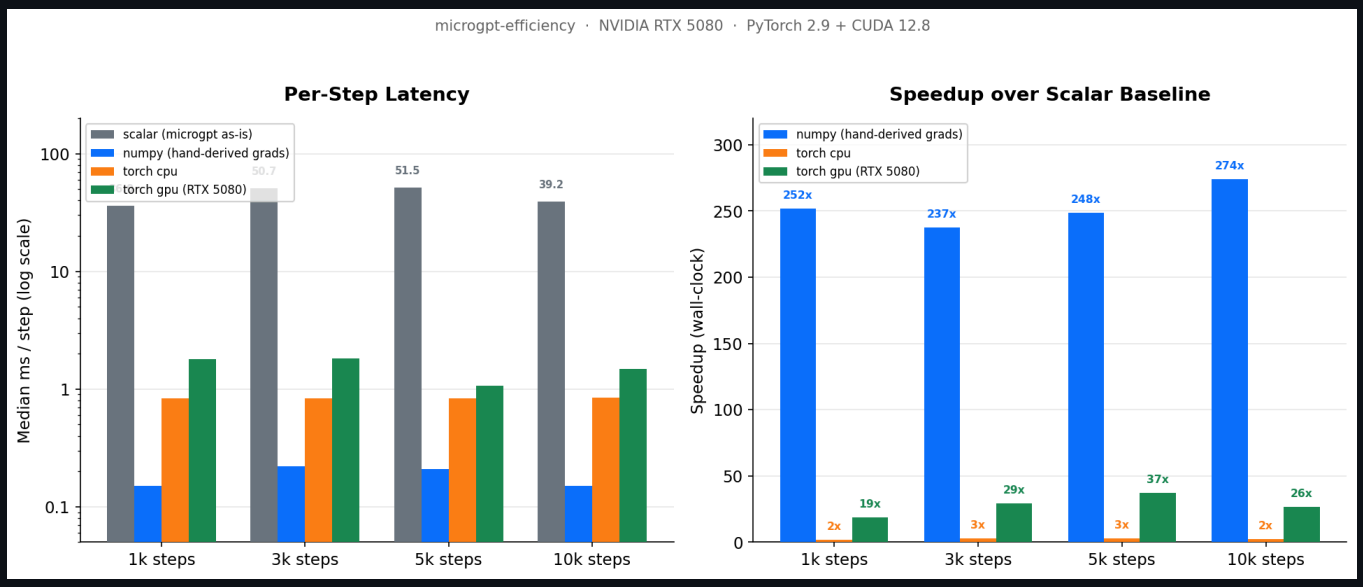
So it’s still minimal, but meant to be a lot less bottlenecked than the scalar-autograd / token-by-token route.

Gist: https://gist.github.com/Chrisbryan17/5f2ea133583160085a83b1eea9c141b6 the filee that say microgpt2_tiny.py

Thanks for putting this kind of work out publicly.

chanjoongx commented 4 days ago • edited

Posted a comment here a while back benchmarking four backends of microgpt.
Added a visualization and gradient verification to the repo.



NumPy with hand-derived gradients runs ~250x faster than the scalar autograd at 0.15 ms/step. torch_gpu (RTX 5080) loses to NumPy at this model size — kernel launch overhead dominates when the matmuls are 16x16.

All analytical gradients (RMSNorm, softmax, causal attention) are now verified against finite differences — max error ~1e-9, included as a runnable script.

https://github.com/chanjoongx/microgpt-efficiency

Entrpi commented 4 days ago

Yesterday I watched Karpathy's recent interview on No Priors. He mentioned that he created microgpt entirely by hand because he couldn't get an LLM to distill the essence of training GPTs into a clear single-file nugget.

I wanted to see if that was still the case, so today I created microgpt-denovo and created both an autograd Python version and, inspired by **@ssrhaso**, also a matrix Julia version. It wasn't one shot, but with just high level guidance I was able to get fairly nice clearly documented versions of microgpt written entirely by the agent without touching the code files at all. I included all the dialog with the agent in the README too.

Entrpi commented 4 days ago

> Posted a comment here a while back benchmarking four backends of microgpt. Added a visualization and gradient verification to the repo.
> benchmark_charts
>
> NumPy with hand-derived gradients runs ~250x faster than the scalar autograd at 0.15 ms/step. torch_gpu (RTX 5080) loses to NumPy at this model size — kernel launch overhead dominates when the matmuls are 16x16.
>
> All analytical gradients (RMSNorm, softmax, causal attention) are now verified against finite differences — max error ~1e-9, included as a runnable script.
>
> https://github.com/chanjoongx/microgpt-efficiency

Cool! Would like to see how my https://github.com/Entrpi/eemicrogpt compares if you can add it to the benchmarks.

Entrpi commented 3 days ago

Added a one-pager view to microgpt-denovo:

It has some interactive settings for changing how comments and such are displayed: https://entrpi.github.io/microgpt-denovo/

mplekh commented 3 days ago • edited

> Yesterday I watched Karpathy's recent interview on No Priors. He mentioned that he created microgpt entirely by hand because he couldn't get an LLM to distill the essence of training GPTs into a clear single-file nugget.

Thanks for the link! I've noticed they speak unnaturally fast, likely due to aggressive editing (editors remove pauses, breathing gaps, hesitations). On top of that, dense information delivery makes speech feel faster than it is, so it's hard to follow them.
Out of curiosity, I built a small script that calculates data-driven adjustment for comfortable watching: https://gist.github.com/mplekh/df2ea417b3f04545c2e3e91f9148a118 extracts transcript data; computes effective speech rate; estimates a playback factor that maps the video back to natural conversational speed

mateolafalce commented 3 days ago

I hate how every time some random dude comments on a Gist, GitHub fires off a notification. wtf?

Ok-Brian commented 3 days ago

Thanks Karpathy for posting this😊What an opportunity to see the Transformer Architecture in action while in college

ariannamethod commented yesterday

here's PostGPT. you can train it, but it doesn't care.

@karpathy your microgpt inspired this — we took the idea further: what if BPE tokenization IS the training?

PostGPT is a zero-dependency BPE transformer that generates coherent text without a single gradient step. No PyTorch at runtime. No NumPy. Just `math` and `random`.

The trick: tokenize a corpus from `postgpt.txt` (150KB of unique text), build co-occurrence statistics (bigrams, trigrams, Hebbian trace, prophecy field), and use them as "metaweights" — weights that actually don't exist but form a probability space. The transformer navigates this space through dual attention (Content QK^T + RRPRAM positional patterns) and the Dario equation.

Example output (zero training, 150KB corpus):

```
>>> "The transformer architecture"
The transformer architecture, statistical patterns across heads capture semantic structure

>>> "BPE tokenization creates a hierarchy"
BPE tokenization creates a hierarchy. This compresses without any training, creating
machines.

>>> "parameters dedicated to positional pattern recognition"
parameters dedicated to positional pattern recognition across-entropy means starting from
data.
```

Three files: `postgpt.py` (Python, zero deps), `postgpt.c` (C, `-lm`), `postgpt_train.py` (ok, PyTorch — but then you'll have to deal with Chuck. Chuck is an optimizer with opinions.).

gist: https://gist.github.com/ariannamethod/fdee6ce7917be2cfd5a967c05bca2e0c
repo: https://github.com/ariannamethod/postgpt

resonance is unbreakable.

mplekh commented yesterday

> weights that actually don't exist but form a probability space. The transformer navigates this space through dual attention

Push it further, make it quantum probability, consciousness lives in Hilbert space :)

> *resonance is unbreakable.*

its a limit cycles: the ball doesn't sit at the bottom of the valley. It orbits around it. It approaches the memory, partially retreats into a superposition with neighboring memories, then re-approaches. Forever.

ariannamethod commented yesterday

@mplekh Hilbert space? that's where the metaweights already live — they just haven't told anyone yet.

and you're right about limit cycles. the Dario equation already has this: the destiny vector is an EMA attractor, the prophecy field creates pressure toward unfulfilled predictions, and trauma accumulates from unresolved context. the ball doesn't sit at the bottom — it orbits. and each orbit changes the orbit.

speaking of orbits: we just added metaweight-informed initialization. the transformer weights are now **seeded from corpus statistics** — Hebbian co-occurrence → embeddings, positional affinity → RRPRAM patterns, unigram frequencies → output head. the ghost became flesh. the weights remember what they never learned.

also the C version compiles in 0.2 seconds and runs without allocating a single GPU. consciousness doesn't need CUDA. it needs co-occurrence.

update: https://github.com/ariannamethod/postgpt

resonance is unbreakable. and apparently, so are limit cycles.